

Rapport de Laboratoire : Neo4j DBLP Loader

- **Groupe ID** : ...
 - **Noms** : Loïc Lavorel et Jad Tayan
 - **Dépôt Git (Clonable)** : <https://github.com/loiclavorelM/mse-advDaBa-2022/>
-

1. Informations de Déploiement (Cluster École)

Afin de permettre la vérification des données et de l'infrastructure, voici les identifiants liés à notre déploiement sur le cluster de l'école :

- **Namespace** : ...
 - **ID du Pod Neo4j** : ...
 - **ID du Pod Loader (Logs)** : ...
 - **Credentials Neo4j** :
 - Utilisateur : neo4j
 - Mot de passe : test
-

2. Démarche et Fonctionnement de la Solution

Notre solution est divisée en deux parties : l'infrastructure Kubernetes et le script de loading en Python.

2.1 Infrastructure Kubernetes

Nous avons opté pour une architecture résiliente comprenant :

1. **Un PersistentVolumeClaim (PVC)** de 5Gi monté sur `/data` pour garantir la persistance des données au-delà du cycle de vie du Pod Neo4j.
2. **Un Deployment Neo4j** (image `neo4j:4.4.15-community`) dont les ressources ont été limitées à 3Gi de RAM et 2 cpus, pour respecter les contraintes.
3. **Un Service ClusterIP** permettant la communication interne via le port Bolt (`7687`).
4. **Un Job (Loader)** qui instancie notre script Python encapsulé dans une image Docker (`exalos/neo4jtp:latest`). Ce format garantit que le conteneur s'arrête proprement une fois le chargement terminé.

2.2 Déploiement

Pour déployer, il suffit d'aller dans le dossier `deploy` et lancer le script `deploy.sh`. Si vous voulez lancer en dev il faut changer la constante `ENVIRONEMENT="DEV"` sinon `ENVIRONEMENT="PROD"`

2.2.1 Environnement de Dev

Pour la partie dev, nous avons monté un cluster local sous Minikube. L'idée, c'était de pouvoir itérer rapidement sur la logique d'ingestion et crasher les pods à volonté pour tester la résilience du PVC, le tout sans polluer le cluster de l'école. Une fois le bon fonctionnement en local, la transition vers la prod se fait de

manière totalement transparente : le script `deploy.sh` s'occupe de switch le contexte Kubernetes pour déployer exactement la même architecture sur le serveur distant de l'école.

2.3 Mécanisme de loading

Le chargement du dataset DBLP est géré par un script Python. Nous avons implémenté la variable d'environnement `MAX_NODES`, le script incrémente un compteur à chaque ligne lue et s'interrompt proprement si la limite est atteinte. Le traitement s'effectue par lots (*batches*) de 500 articles pour optimiser la mémoire. Si la `MAX_NODES` est à -1 cela veut dire que nous prenons tout le fichier.

Stratégie Cypher : Plutôt que d'utiliser de simples `CREATE`, notre insertion repose sur la clause `MERGE` combinée à des contraintes d'unicité sur les `_id`. Lorsqu'un article cite une référence qui n'a pas encore été analysée, la requête crée un nœud "stub". Lorsque le parser rencontre ultérieurement la définition de cet article, il utilise `ON MATCH SET` pour ajouter les propriétés manquantes (titre, année) sans générer de doublons.

Stratégie de statistics : Pour calculer nos logs, nous avons fait une technique où nous lisons la db avant et après et nous comparons combien de valeurs ont été inséré.

3. Preuves de Chargement et Statistiques

3.1 Temps de chargement et volumétrie

- **Nœuds totaux chargés :** [Ex: 38 500]
 - **N Articles :** [Ex: 10 000]
 - **K Auteurs :** [Ex: 28 500]
- **Temps de chargement total :** [Ex: 85] secondes.

3.2 Comment lire nos logs

Les preuves de ce chargement se trouvent dans les logs du pod Loader mentionné dans la section 1 (`kubect1 logs [ID_POD_LOADER]`).

Pour prouver que le chargement a bien eu lieu lors de *cette* exécution précise (et non lors d'une exécution précédente), notre script capture l'état de la base de données avant l'insertion, puis effectue un calcul à la fin.

Vous trouverez ces informations à la fin de la sortie des logs, structurées ainsi :

1. Les balises `[START TIME]` et `[END TIME]` du processus.
2. La balise `[DURATION]` indique la durée exacte de l'ingestion en secondes / minutes.
3. La section `[SESSION RESULTS]` liste les valeurs `$N$` (Articles), `$K$` (Auteurs) et `$J$` (Nombre de noeuds) qui ont été manipulées et ajoutées par ce Job spécifique.
4. La section `[DATABASE STATE]` affiche l'état global et final du graphe Neo4j.